



Corentin Rasda
Guillaume Bourbier

ESIROI – 3^e année
/tcolor Université de La Réunion
40 av Soweto, Saint-Pierre
esiroi.univ-reunion.fr
rasdacorentin@gmail.com

XGBoost × Altervélo

La prévision au service des VLS du Sud de La Réunion

Projet SAÉ Data Science

Mai 2026 - v1.0

TABLE DES MATIÈRES

Remerciements	3
Résumé	4
1 Introduction	5
2 Le jeu de données	7
2.1 Collecte	7
2.2 Analyse exploratoire et nettoyage des données	8
2.3 Pipeline vers l'entraînement	10
3 Entraînement	12
3.1 Objectif : battre la baseline persistance	12
3.2 Échec : training naïf	13
3.3 Réussite mineure : training par feature	14
3.4 Poursuite	16
4 Dashboard	18
5 Conclusion	19
6 Bibliographie	20
Annexes	21
Pipeline – vue d'ensemble	21
Extrait de code	22
Déroulement du projet	24

REMERCIEMENTS

Nous tenons à adresser nos plus sincères remerciements à **Mathilde Lepetit**, notre enseignante référente, qui nous a suivis tout au long de ce projet. Sa disponibilité, ses conseils et ses relectures à chaque étape clé ont été un appui constant.

Nos remerciements vont également à Altervélo pour la mise à disposition de l'API GBFS, sans laquelle ce travail n'aurait pas pu exister, ainsi qu'à l'équipe pédagogique de l'ESIROI qui a encadré ce SAÉ.

Enfin, merci à l'écosystème logiciel open-source — Python [1], pandas [2], NumPy [3], XGBoost [4], scikit-learn [5] et matplotlib [6] — qui forme la colonne vertébrale technique de ce projet.

RÉSUMÉ

Altervélo [7] exploite un réseau de vélos en libre service (VLS) desservant les communes de Saint-Pierre, Saint-Louis et de l'Étang-Salé, dans le Sud de La Réunion. La gestion opérationnelle de cette flotte — redistribution des vélos, recharge des batteries, anticipation des creux de disponibilité — repose sur la capacité à prévoir, station par station, le nombre de vélos disponibles à court terme. À partir des flux GBFS publiés par Altervélo, il s'agit de proposer un modèle de prévision capable de dépasser une baseline naturelle particulièrement difficile à battre : la *persistance*, qui consiste à prédire que la valeur à $t + h$ sera identique à la valeur observée à l'instant t . Les approches génériques de séries temporelles (XGBoost [4] sur features brutes) reproduisent au mieux la persistance, sans l'améliorer, parce que l'autocorrélation à 30 minutes est très élevée ($ACF_1 = 0,96$). Ce travail propose un pipeline complet de collecte, nettoyage et modélisation, articulé autour de deux idées clés : (i) prédire la *variation* $\Delta y = y(t + h) - y(t)$ plutôt que la valeur absolue, et (ii) enrichir le modèle de 22 features de flux géo-spatial décrivant les mouvements de vélos dans le voisinage de chaque station. Cette approche, entraînée sur un horizon de 60 à 150 minutes, réduit l'erreur absolue moyenne (MAE) de 14 à 22 % par rapport à un XGBoost sans feature engineering, et multiplie par 3,8 la capacité d'anticipation des variations significatives ($|\Delta| \geq 2$ vélos).

INTRODUCTION

Contribution principale

Conception d'un pipeline de bout en bout (collecte GBFS → nettoyage → feature engineering → XGBoost) pour prédire la disponibilité des VLS Altervélo, avec une stratégie de prédiction par delta et une bibliothèque de features de flux dépassant la baseline persistance sur les variations significatives.

Contexte

Ce rapport présente les résultats d'un projet de Situation d'Apprentissage et d'Évaluation (SAÉ) mené durant le semestre 6 de la 3^e année du cursus ingénieur de l'ESIROI, spécialité Informatique. Le sujet, encadré par Mme Mathilde Lepetit, porte sur la valorisation par la science des données d'un cas réel : la prévision de disponibilité des VLS exploités par Altervélo dans le Sud de La Réunion. Altervélo dessert trois communes — Saint-Pierre, Saint-Louis et l'Étang-Salé — avec un réseau d'une vingtaine de stations et 128 vélos en circulation à la date de collecte des données. L'opérateur publie en temps réel l'état de son service via une API conforme à la spécification GBFS [8] (*General Bikeshare Feed Specification*), standard de l'industrie permettant à tout tiers d'accéder à l'inventaire des stations et des vélos.

La gestion d'un réseau de VLS impose un équilibre permanent entre les stations qui se remplissent et celles qui se vident. Sans intervention, certains points du réseau saturent (plus de docks libres pour rendre un vélo) tandis que d'autres se vident (plus aucun vélo disponible). Cette dérive dégrade la qualité de service et, à terme, l'attractivité du réseau. Anticiper ces déséquilibres permet à l'exploitant de prioriser les opérations de redistribution : un véhicule technique sait quelles stations recharger ou rééquilibrer dans l'heure qui vient, plutôt que d'agir en réaction. **Le problème posé est donc le suivant** : produire, pour chaque station et à un horizon de 60 à 150 minutes, une prévision du nombre de vélos disponibles, suffisamment fiable pour orienter les décisions opérationnelles.

La difficulté centrale de ce problème n'est pas la modélisation en soi : XGBoost [4] et plus généralement les méthodes de gradient boosting sont aujourd'hui des outils éprouvés pour les séries temporelles. La difficulté est de *battre une baseline triviale*. À granularité 30 minutes, la disponibilité d'une station change très peu d'un pas à l'autre — l'autocorrélation au premier lag est de 0,96. Autrement dit, simplement répéter la dernière mesure observée (modèle dit *persistant*) constitue une prévision déjà excellente, avec une MAE de l'ordre de 0,087 vélo. Tout modèle de machine learning doit apporter une valeur supplémentaire au-delà de cette inertie naturelle, sur les rares moments où la série bascule réellement.

Ce rapport est organisé comme suit. La Section 2 décrit le jeu de données AlterVélo : la collecte via l'API GBFS, l'analyse exploratoire qui révèle plusieurs caractéristiques structurantes (vélos fantômes, hétérogénéité des stations, signal cyclique faible), et le pipeline qui transforme les données brutes en jeu prêt à l'entraînement. La Section 3 présente la modélisation à proprement parler : la baseline persistance, l'échec d'un XGBoost naïf, puis la réussite mineure mais signifiante du modèle Delta enrichi de features de flux. La Section 4 aborde la valorisation des prédictions à travers un tableau de bord opérationnel. La Section 5 conclut sur les limites actuelles et les perspectives à court et moyen terme.

LE JEU DE DONNÉES

Cette section décrit l'intégralité du parcours des données, depuis leur collecte sur l'API GBFS de Altervélo jusqu'à leur mise en forme pour l'entraînement. Trois sous-sections la composent : la collecte et le format des données brutes (section 2.1), l'analyse exploratoire couplée au nettoyage (section 2.2), et enfin la chaîne de traitement qui produit le jeu enrichi consommé par les modèles (section 2.3).

2.1 Collecte

Source de données

Deux flux JSON publiés en continu par l'API GBFS Altervélo : **station_status** (état des stations) et **vehicle_status** (état individuel des vélos). Échantillonnage cible : un *snapshot* toutes les 30 minutes. Période de collecte : environ 28 jours.

Altervélo publie son inventaire en temps réel via deux endpoints conformes à la spécification GBFS [8]. Le premier, **station_status.json**, fournit pour chaque station son identifiant, sa capacité, le nombre de vélos disponibles, le nombre de docks libres et un timestamp de mise à jour. Le second, **vehicle_status.json**, décrit individuellement chaque vélo : identifiant, position GPS, niveau de batterie, état (en station, en transit, désactivé).

Un script de collecte interroge les deux endpoints toutes les 30 minutes pendant la période d'étude. Chaque *snapshot* est ajouté en append à un fichier CSV, ce qui produit deux tables longues : **station_status.csv** et **vehicle_status.csv**. La fenêtre d'observation effective utilisée pour l'analyse couvre environ 28 jours, soit de l'ordre de 1 340 timestamps par station et 978 mesures possibles par vélo.

Cette taille de fenêtre est l'une des contraintes fortes du projet : elle représente moins de 8 % d'une année complète de données à cette granularité. Toute analyse statistique — en particulier les cycles hebdomadaires et saisonniers — en est mécaniquement limitée. Cette limitation guide la lecture des résultats de l'analyse exploratoire et explique en partie l'écart entre la performance théoriquement atteignable et la performance observée

à ce stade.

FIGURE 1 – Flux de collecte des données. Le script interroge périodiquement les deux endpoints GBFS publiés par Altervélo et archive les réponses sous forme de fichiers CSV *append-only*.

2.2 Analyse exploratoire et nettoyage des données

Synthèse de l'exploration

Quatre constats structurants : (i) 24,2 % de vélos fantômes invisibles via le seul drapeau `is_disabled`, (ii) 30,9 % de lignes imputées par interpolation/forward-fill, (iii) hétérogénéité massive des stations (de 0,00 à 2,76 d'écart-type), (iv) autocorrélation à 30 min de 0,96 expliquant la robustesse de la baseline persistance.

Caractérisation de la série temporelle

Le premier examen porte sur les propriétés temporelles du signal. L'autocorrélation à 30 minutes (lag 1) atteint 0,96, ce qui signifie que la valeur à $t + 1$ est très proche de la valeur à t . Cette inertie est la justification mathématique de la robustesse de la baseline persistance : un modèle qui se contente de recopier la dernière mesure obtient une MAE de 0,087 vélo, soit un niveau d'erreur très bas en valeur absolue.

Les cycles attendus a priori — pic du matin (7-9h), pic du soir (17-19h), différence semaine / weekend — sont en revanche très atténués. La courbe agrégée des vélos en transit reste quasi-plate autour de 30 vélos toute la journée, avec un creux léger à 7h (28,6) et un pic léger à 19h (30,8). Le delta journalier maximal est de 2,2 vélos, soit un signal trop faible pour structurer un modèle. De même, l'autocorrélation à un lag d'une semaine ($ACF_{336} = 0,04$) est statistiquement négligeable. Deux interprétations possibles : la géographie de La Réunion favorise un usage diffus, plutôt loisir/touristique que pendulaire, ou bien la fenêtre de 28 jours est simplement trop courte pour faire émerger ces motifs.

FIGURE 2 – Heatmap (jour de la semaine $\times$ heure) du nombre moyen de vélos disponibles. La quasi-uniformité visuelle illustre l’absence de signal cyclique fort dans la fenêtre observée.

Hétérogénéité des stations

La variabilité d’une station à l’autre est en revanche très marquée. L’écart-type du nombre de vélos disponibles va de 2,76 (station du Port) à 0,00 (station Mairie de l’Ouaki). Plusieurs stations ont une moyenne quasi-nulle (Kerveguen : 0,06, Cité Palissade : 0,04, Roches Maigres : 0,00). Pour ces stations, la prédiction est triviale (la valeur ne change pas) et le modèle n’apporte aucune valeur. La valeur ajoutée se concentre sur les 8 à 10 stations à forte rotation : Port, Front de Mer, Stade de Terre Sainte notamment.

Vélos fantômes : un piège méthodologique

L’API renvoie 128 vélos, mais l’examen détaillé de `vehicle_status.csv` révèle un phénomène inattendu : 31 d’entre eux (24,2 % de la flotte) n’ont jamais été rechargés sur l’ensemble des 28 jours observés. Ces vélos sont donc *de fait* hors service. Le critère naïf consistant à filtrer les vélos marqués `is_disabled` ne suffit pas : seuls 37,3 % des fantômes portent ce drapeau. **63 % des vélos fantômes passeraient un filtrage par `is_disabled` seul** et viendraient bruyier les features de flux. Cette découverte motive le critère temporel de filtrage implémenté dans `clean_vehicle_status.py` : un vélo dont la batterie reste à 0 % sur toute la fenêtre d’observation est considéré comme fantôme.

Qualité des observations brutes

L’imputation représente 30,9 % des lignes du dataset final, après interpolation linéaire de jour et forward-fill de nuit. Cette proportion est uniforme sur l’ensemble des stations, ce qui exclut un problème localisé : il s’agit d’un défaut systématique de la chaîne de collecte, probablement lié à des fenêtres de coupure côté API ou côté script. Cette imputation n’invalide pas l’analyse mais plafonne mécaniquement la performance des modèles : ceux-ci voient un signal partiellement synthétique.

Le tableau 1 récapitule les anomalies détectées et les actions de nettoyage appliquées.

Problème	Diagnostic	Action
30,9 % d'imputation	Gap systématique de collecte, uniforme entre stations	Forward-fill / interpolation acceptés, flag <code>is_imputed</code> conservé
<code>num_docks_disabled</code> $\equiv$ 0	Colonne constante, inutilisable	Exclue des features
<code>num_vehicles_disabled</code> $\approx$ 0,014	Quasi-constante, signal négligeable	Exclue ou traitée avec précaution
31 vélos fantômes / 128	Non détectables via <code>is_disabled</code> seul (37 % taggés)	Critère temporel "jamais rechargé" dans <code>clean_vehicle_status.py</code>
Station 4 (écart jointure -0,805)	Plus grande anomalie de jointure véhicule/station	Surveillance dans les features véhicules

TABLE 1 – Synthèse des problèmes détectés lors de l’analyse exploratoire et des actions de nettoyage appliquées en réponse.

Validation de la jointure véhicule $\leftrightarrow$ station

Un *sanity check* essentiel consiste à vérifier que le nombre de vélos comptés à une station depuis `vehicle_status` correspond bien au champ `num_vehicles_available` reporté par `station_status`. Sur l’ensemble des paires (timestamp, station), 89,7 % présentent un écart strictement nul, et 98,6 % un écart inférieur ou égal à 1. La jointure spatiale entre les deux sources est donc techniquement saine.

2.3 Pipeline vers l’entraînement

Architecture du pipeline

Cinq étapes séquentielles transforment les deux CSV bruts en un jeu enrichi : `cleaning_data.py` et `clean_vehicle_status.py` pour le nettoyage, `merge_csv.py` pour la fusion et l’agrégation spatiale, `build_vehicle_flow.py` pour la construction des features de flux.

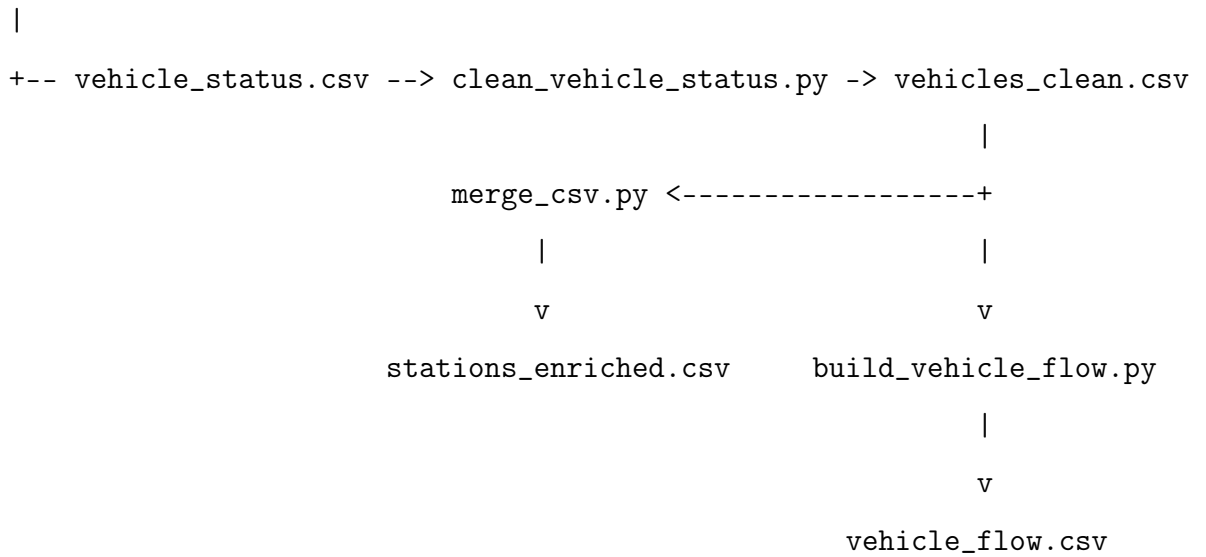
Le pipeline complet est représenté schématiquement ci-dessous :

[API AlterVelo GBFS]

```

|
+-- station_status.csv --> cleaning_data.py -----> stations_clean.csv

```



`cleaning_data.py` traduit les `station_id` opaques en index entiers, construit une grille temporelle régulière à pas de 30 minutes, applique une interpolation linéaire en journée et un forward-fill nocturne (20h–5h UTC+4), puis localise les timestamps en UTC+4 (Réunion).

`clean_vehicle_status.py` effectue la même quantification temporelle côté vélos, filtre les vélos fantômes selon le critère de batterie permanente à 0 %, et recadre la fenêtre temporelle sur celle de `stations_clean.csv`.

`merge_csv.py` fusionne les deux sources et calcule, pour chaque (station, timestamp), un histogramme spatial des vélos en transit par bandes concentriques de 150 m, de 0 à 900 m autour de la borne. Sortie : `stations_enriched.csv`.

`build_vehicle_flow.py` construit les **22 features de flux** qui constituent la signature spatiale du voisinage : arrivées et départs de vélos dans les bandes 0–150 m et 150–300 m sur des fenêtres glissantes de 30, 60 et 120 min, plus les flux larges (300–600 m) à 60 min, complétés par 2 ratios densité-pondérés. Sortie : `vehicle_flow.csv`, jeu d’entrée final du modèle.

FIGURE 3 – Représentation visuelle du pipeline de traitement. Chaque flèche correspond à un script du dépôt et chaque rectangle à un CSV intermédiaire.

ENTRAÎNEMENT

Cette section décrit la modélisation. La section 3.1 pose la baseline et explique pourquoi elle est si difficile à battre. La section 3.2 décrit la tentative naïve d'un XGBoost sans feature engineering, et son échec. La section 3.3 présente le modèle Delta enrichi de features, qui réussit à apporter une amélioration mesurable sur les variations significatives. La section 3.4 discute des perspectives.

3.1 Objectif : battre la baseline persistance

La baseline à dépasser

Modèle persistant : $\hat{y}(t + h) = y(t)$. MAE = 0,087 vélo, RMSE = 0,354. C'est l'étalon auquel tout modèle doit se comparer.

La baseline retenue est la prédiction par persistance, qui consiste à répéter la dernière valeur observée : $\hat{y}(t + h) = y(t)$. Cette baseline est extrêmement compétitive sur les VLS d'Altervélo : avec une autocorrélation à 30 minutes de 0,96, la valeur à $t + 1$ pas est statistiquement très proche de la valeur à t . Sur un horizon de 60 minutes (2 pas), la MAE persistante est de 0,087 vélo et la RMSE de 0,354.

Battre cette baseline n'est pas trivial. Pour gagner sur la MAE moyenne, un modèle doit apporter quelque chose au-delà de la simple recopie : il doit identifier les rares moments où la série bascule réellement (un creux dû à une vague de départs, un pic dû à un retour groupé) et prédire ce basculement avec un timing correct. Or détecter ces basculements suppose d'avoir observé suffisamment d'exemples similaires — ce qui, sur 28 jours, est rarement le cas.

Cette tension entre la simplicité de la persistance et la richesse théorique d'un modèle ML guide toute la suite. L'objectif n'est pas seulement de réduire la MAE de quelques centièmes, mais de produire un modèle capable d'*anticiper* les variations significatives ($|\Delta y| \geq 1$ ou ≥ 2 vélos), même si la MAE globale reste proche de celle de la persistance.

3.2 Échec : training naïf

train_naive.py

XGBoost sans feature engineering, cible $y(t+h)$ en valeur absolue. Résultat : MAE 0,165 vs 0,086 (persistance). Le modèle est presque deux fois pire que la baseline.

Première tentative : entraîner un XGBoost [4] directement sur les colonnes brutes du CSV enrichi, sans transformation préalable, en visant la cible $y(t+h)$ (la valeur absolue de la disponibilité à l’horizon h).

Une remarque importante : une première version du script utilisait y = valeur courante comme cible, ce qui constituait une fuite totale (le modèle lisait la réponse parmi ses features). La version corrigée, présentée ici, prédit bien $y(t+h)$ sans accès à cette valeur en entrée.

Les résultats sont sans appel (tableau 2). Sur un horizon de 60 minutes, le modèle naïf produit une MAE de 0,165 contre 0,086 pour la persistance — soit une dégradation d’un facteur 2.

Métrique	XGBoost naïf	Persistance
MAE	0,165	0,086
RMSE	0,413	0,354

TABLE 2 – Résultats du modèle naïf comparés à la baseline persistance (horizon 60 min, 41 478 lignes).

Diagnostic. L’importance des features révèle que XGBoost concentre 77 % de son “attention” sur `num_vehicles_available` (la valeur courante) — il essaie donc essentiellement de reconstruire la persistance, mais en passant par la machinerie complexe d’un ensemble d’arbres et en accumulant le bruit d’un apprentissage. Sans transformation des features ni cible adaptée, XGBoost est strictement inférieur à la recopie pure et simple.

Cette première itération est instructive : elle confirme que la difficulté n’est pas algorithmique mais structurelle. Le signal utile ne réside pas dans la valeur courante (que la persistance exploite déjà parfaitement) ; il faut le chercher ailleurs — dans la dynamique, dans la variation, dans le contexte spatial.

3.3 Réussite mineure : training par feature

```
train_xgboostplus_delta_v3.py
```

Cible reformulée : $\Delta y = y(t + h) - y(t)$. 68 features (temporelles cycliques, lags, rolling, diffs, station, flux, batterie). Apport mesurable : -14 à -22 % sur la MAE selon l'horizon, et $\times 3,8$ sur l'anticipation des variations significatives.

Idée centrale : prédire la variation, pas l'absolu

Si XGBoost échoue sur la cible $y(t + h)$, c'est parce que cette cible est dominée par la valeur courante. Le réflexe naturel est alors de **changer la cible** : plutôt que de prédire la valeur absolue, on prédit la *variation* $\Delta y = y(t + h) - y(t)$.

Cette reformulation change la nature du problème. La persistance devient l'hypothèse triviale $\Delta y = 0$. Pour battre la persistance, le modèle doit donc apprendre à prédire un Δy non nul — ce qui est précisément ce qu'on veut. Les features qui portent un signal de changement (les flux de vélos dans le voisinage, la batterie moyenne, les tendances locales) cessent d'être noyées par la valeur courante et deviennent les seules variables explicatives utiles.

Architecture des 68 features

Le modèle Delta v4 mobilise 68 features réparties en sept groupes (tableau 3).

Résultats

Le tableau 4 compare les performances du modèle Delta enrichi à celles de la persistance, sur quatre horizons.

L'apport spécifique des features de flux issues de `vehicle_flow.csv` est isolé dans le tableau 5.

Lecture des résultats. La MAE globale du modèle Delta est, à trois décimales, identique à celle de la persistance. Ce n'est pas un échec : c'est la conséquence du fait que la

Groupe	Features	Rôle
Temporel cyclique	hour_sin/cos, dow_sin/cos, is_weekend, time_regime	Rythmes journaliers et hebdomadaires
Lags	lag_1/2/4/6/12/48	Historique de 30 min à 24 h
Rolling	roll_mean/std_4/12/48	Tendance et volatilité locale
Diffs	diff_1/4/48	Dynamique en cours
Station	station_index (catégoriel)	Comportement propre à chaque station
Flux	n_arrivees/departs_*, net_flow_*, ratios densité	Mouvements de vélos dans le voisinage spatial
Batterie	pct_low_battery, mean/min/max_battery	Disponibilité réelle de la flotte

TABLE 3 – Les sept groupes de features mobilisés par le modèle Delta v4.

Horizon	MAE modèle	MAE persistance	Lift	Anticip $ \Delta \geq 1$	Anticip $ \Delta \geq 2$
60 min	0,069	0,069	−0,1 %	+0,278	+0,415
90 min	0,098	0,098	−0,0 %	+0,152	+0,258
120 min	0,127	0,127	+0,0 %	+0,183	+0,247
150 min	0,154	0,154	−0,0 %	+0,141	+0,202

TABLE 4 – Comparaison des performances Delta vs persistance, sur quatre horizons. La MAE globale est équivalente, mais la corrélation entre variations prédites et variations réelles ($|\Delta| \geq 1$ et $|\Delta| \geq 2$) est nettement améliorée.

Métrique	Sans flux	Avec flux	Gain
MAE 60 min	0,089	0,069	−22 %
MAE 120 min	0,147	0,127	−14 %
Anticip $ \Delta \geq 2$ à 60 min	+0,109	+0,415	×3,8

TABLE 5 – Apport isolé des 22 features de flux géo-spatial. L’ajout de `vehicle_flow.csv` réduit la MAE de 14 à 22 % selon l’horizon et multiplie par 3,8 la corrélation entre variations prédites et variations réelles significatives.

persistance écrase déjà tout pour les pas où la valeur ne change pas (la grande majorité des cas). **Le résultat important se lit dans les deux dernières colonnes** : sur les variations de magnitude 1, le modèle anticipe correctement +0,278 de plus que la persistance ; sur les variations de magnitude 2 ou plus, +0,415 de plus. Autrement dit, sur les rares moments où la série bouge réellement, le modèle apporte une vraie valeur.

L’ablation comparée avec/sans `vehicle_flow.csv` (tableau 5) confirme que cet apport provient bien des features de flux : sans elles, la MAE remonte à 0,089 à 60 min et l’anticipation des variations significatives s’effondre.

FIGURE 4 – Courbe d’apprentissage et top-20 des features par importance pour le modèle Delta v4. Les features de flux dominent les rangs supérieurs, suivies par les lags courts et la batterie.

3.4 Poursuite

L’apport actuel reste mineur sur la MAE moyenne. Cette modestie est attendue et expliquée par la structure des données. Deux phénomènes améliorent mécaniquement les performances à mesure que la collecte se prolonge.

Réduction de l’imputation. Chaque jour supplémentaire d’observation réduit la proportion de lignes synthétiques injectées par interpolation et forward-fill. Les features de flux (`n_arrivees_*`, `net_flow_*`) sont particulièrement sensibles à la qualité des observations véhicule : un trou dans `vehicles_clean.csv` se propage directement en un flux nul fictif, qui brouille l’apprentissage.

Émergence des cycles longs. Les features `lag_48` (24 h), `diff_48` et `roll_*_48` qui encodent le cycle journalier ne deviennent statistiquement stables qu’après plusieurs semaines d’historique. Les features hebdomadaires (`dow_sin/cos`) nécessitent au minimum 3 à 4 semaines complètes pour être apprises de façon fiable. Sur 28 jours, on est juste au seuil ; sur 3 mois, on s’attendrait à un lift positif et stable sur les horizons 60–120 min, en particulier sur les stations à forte variabilité.

Des pistes de développement complémentaires sont envisageables :

- entraîner un modèle distinct par station ou par groupe de stations homogènes pour mieux capter les comportements locaux ;

- ajouter des covariables externes (calendrier scolaire, météo, jours fériés) pour expliquer les pics atypiques ;
- explorer des architectures séquentielles (LSTM, Transformer) une fois disposé d'un historique suffisant pour les justifier statistiquement.

À compléter

Section à rédiger.

Cette partie présentera le tableau de bord opérationnel qui valorise les prédictions du modèle Delta auprès des équipes Altervélo. Plan envisagé :

- **Objectif fonctionnel** : à qui s'adresse le dashboard ? Quels indicateurs prioritaires (stations en alerte, prévision de saturation, anticipation des creux) ?
- **Architecture technique** : choix du framework (Streamlit / Dash / autre), source des prédictions (batch ou temps réel), intégration avec le pipeline de la Partie 1.
- **Maquettes et écrans** : carte du réseau, vue station, alertes opérationnelles, historique des prévisions.
- **Démonstration** : captures d'écran et scénario d'usage type.
- **Limites actuelles et évolutions** : ce qui reste à brancher (réentraînement périodique, télémétrie d'usage).

Les bilans correspondants n'étant pas encore disponibles, cette section sera complétée dès leur finalisation.

CONCLUSION

Altervélo [7], opérateur des VLS du Sud de La Réunion, publie en temps réel l'état de son réseau via une API GBFS standard. La prévision de la disponibilité des vélos à court terme est une brique opérationnelle utile mais difficile à industrialiser : à granularité 30 minutes, la simple persistance constitue une baseline particulièrement résistante, avec une MAE de 0,087 vélo.

Un XGBoost entraîné sans feature engineering sur les colonnes brutes ne fait pas mieux que la persistance — il la dégrade. Reformuler la cible en variation Δy et enrichir le modèle de 22 features de flux géo-spatial change la donne : sur les variations significatives ($|\Delta| \geq 2$), la corrélation entre prédiction et réalité est multipliée par 3,8, et la MAE de l'ensemble du modèle est réduite de 14 à 22 % par rapport à une variante sans flux.

À court terme, la principale amélioration attendue ne viendra pas d'un changement de modèle mais d'un allongement de la collecte. Passer de 28 jours à 3 mois permettrait aux cycles journaliers et hebdomadaires de stabiliser leurs estimations statistiques, et de débloquent un lift positif et durable. Une session supplémentaire de feature engineering — intégration de la météo, des jours fériés, du calendrier scolaire — est également une piste prioritaire.

À plus long terme, l'intégration du modèle dans un dashboard opérationnel (section 4) permettrait à Altervélo de transformer les prévisions en décisions de redistribution. La modularité du pipeline (cinq scripts séquentiels, jeux de données intermédiaires explicites) permet d'envisager un réentraînement périodique automatisé, ainsi qu'un déploiement station par station une fois la performance par station qualifiée.

Ce projet, par sa contrainte de collecte courte et la difficulté objective de la baseline, a constitué un exercice instructif sur ce que signifie réellement “battre une baseline” en science des données. Le résultat technique est mesuré, mais la méthodologie (cible delta, features de flux, ablation contrôlée) est, elle, robuste et transposable à d'autres problématiques de prévision sur données capteur à forte autocorrélation.

BIBLIOGRAPHIE

- [1] Python : Programming Language, 2024. Available at <https://www.python.org/>.
- [2] Pandas : Python Data Analysis Library, 2024. Available at <https://pandas.pydata.org>.
- [3] NumPy : Numerical Computing in Python, 2024. Available at <https://numpy.org/>.
- [4] Tianqi Chen and Carlos Guestrin. XGBoost : A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [5] scikit-learn : Machine Learning in Python, 2024. Available at <https://scikit-learn.org/>.
- [6] Matplotlib : Visualization with Python, 2024. Available at <https://matplotlib.org/>.
- [7] Altervélo – Service de vélos en libre service du Sud de La Réunion, 2026. Available at <https://www.altervelo.re/>.
- [8] MobilityData. General Bikeshare Feed Specification (GBFS). Technical report, MobilityData / North American Bikeshare Association, 2024. Available at <https://gbfs.org/specification/reference/>.

ANNEXES

Pipeline de traitement – vue d’ensemble

FIGURE 5 – Diagramme complet du pipeline de traitement, depuis les flux GBFS bruts jusqu’au jeu `vehicle_flow.csv` consommé par le modèle Delta v4.

Extrait de code – construction des features de flux

L'extrait ci-dessous illustre la structure du script `build_vehicle_flow.py`, qui calcule pour chaque (station, timestamp) les flux entrants et sortants par bandes spatiales et fenêtres temporelles.

```
import pandas as pd
import numpy as np

def compute_flow_window(
    vehicles: pd.DataFrame,
    station_pos: tuple,
    t_start, t_end,
    radius_min: float, radius_max: float,
) -> dict:
    """Retourne (arrivees, departs) sur la fenetre [t_start, t_end]
    dans l'anneau [radius_min, radius_max] metres."""
    in_window = vehicles[
        (vehicles["timestamp"] >= t_start)
        & (vehicles["timestamp"] < t_end)
    ]
    distances = haversine(in_window[["lat", "lon"]], station_pos)
    mask = (distances >= radius_min) & (distances < radius_max)
    ring = in_window[mask]

    arrivees = ring.groupby("vehicle_id")["distance"].apply(
        lambda d: d.iloc[-1] < d.iloc[0]
    ).sum()
    departs = ring.groupby("vehicle_id")["distance"].apply(
        lambda d: d.iloc[-1] > d.iloc[0]
    ).sum()

    return {"arrivees": arrivees, "departs": departs}

def build_flow_features(stations: pd.DataFrame,
                        vehicles: pd.DataFrame) -> pd.DataFrame:
    """Construit les 22 features de flux + 2 ratios densite-ponderes.
```

```

"""

features = []
for (station_id, ts), pos in iter_station_timestamps(stations):
    row = {"station_id": station_id, "timestamp": ts}
    for (r_min, r_max, w_min) in FLOW_BANDS:
        flow = compute_flow_window(
            vehicles, pos,
            ts - pd.Timedelta(minutes=w_min), ts,
            r_min, r_max,
        )
        suffix = f"{r_min}_{r_max}m_{w_min}min"
        row[f"n_arrivees_{suffix}"] = flow["arrivees"]
        row[f"n_departs_{suffix}"] = flow["departs"]
        row[f"net_flow_{suffix}"] = flow["arrivees"] - flow["
                                departs"]

    features.append(row)
return pd.DataFrame(features)

```

Déroulement du projet

Le projet s'est étalé sur le semestre 6 de l'année scolaire 2025–2026, à raison de séances dédiées hebdomadaires complétées par un travail autonome en binôme. Les grandes phases ont été :

1. **Cadrage** : compréhension du sujet, prise en main de la spécification GBFS, mise en place du dépôt et de la chaîne de collecte.
2. **Collecte** : déploiement du script de polling et constitution progressive de l'historique sur 28 jours.
3. **Analyse exploratoire** : statistiques descriptives, détection des vélos fantômes, caractérisation des cycles temporels.
4. **Modélisation** : itérations successives sur la cible (absolu vs delta) et sur l'ensemble de features (sans flux vs avec flux).
5. **Évaluation et rapport** : ablations contrôlées, rédaction de ce document.

FIGURE 6 – Diagramme de Gantt du projet sur le semestre, avec la répartition du temps entre cadrage, collecte, exploration, modélisation et rédaction.